Data & AI Disclosure

DVN36104 Data Visualisation and Narratives — Assignment 3
Group 18

Data Disclosure

This project uses public data from **Epoch AI**.

The main data sources are:

- **Epoch AI — Notable AI Models**, used for the macro analysis of training-compute growth.
- **Epoch AI — Capabilities & Benchmarking**, used for GPQA Diamond capability analyses.

The original source exports are retained in the repository alongside processed datasets. Data preparation is performed through `prepared_data.py`.

The project does not intentionally fabricate or impute missing benchmark scores or training-compute values. Observations without the required measured values are excluded where relevant to the analysis.

Detailed information about source URLs, licences, variables, units, time coverage and transformations is documented in:

`data/DATA_DICTIONARY.md`

Generative AI Disclosure

Generative AI tools were used as development assistants during this project.

AI assistance was used for activities including:

- debugging Python and Streamlit code;
- reviewing dashboard layout and information hierarchy;
- discussing visualisation alternatives;
- refining explanatory text and documentation; and
- supporting code review and troubleshooting.

AI-generated suggestions were not treated as authoritative analytical results. Team members remained responsible for the analytical choices, source-data interpretation, final code and claims presented in the application.

Where AI-assisted code or analysis was used, outputs were checked by running the application, examining source or processed data, and comparing displayed results with independently recalculated values.

The table below records substantive AI use by each team member and the verification performed.

AI Provenance Matrix

Team member	AI tool	Substantive use	Where used	AI contribution	Human verification	Outcome
				Used ChatGPT to assist with Streamlit debugging, review the		

Nguyen Thao Nhi Trường	ChatGPT	Development and review of User Story 2: capability at comparable compute	`tabs/tab3.py`; project framing and documentation	layout and information hierarchy of User Story 2, discuss the choice of a yearly-median line chart rather than an individual-model scatter plot, and refine explanatory wording. ChatGPT also assisted with restructuring the tab so the controls, KPIs, main visual, verification and limitations followed a consistent dashboard layout.	The code was run and inspected using the actual processed Epoch AI GPQA data. For the default **10^{24}–10^{25} FLOP** band, the displayed **2026 median GPQA value of 89.74%** was independently recalculated from the **10 underlying observations**. The verification also tested that every observation fell inside the selected compute band and that the displayed observation count matched the underlying rows. These checks could have failed if the filtering, aggregation or AI-assisted code were incorrect.	The recalculated median matched the displayed value, all observations satisfied the selected compute-band rule, and the displayed count matched the underlying rows. The AI-assisted implementation was retained after these checks.
Yuchao Zong	Claude (Anthropic)	Development of the macro overview: training-compute growth	tabs/tab1.py; Macro Overview section of README.md	Claude wrote most of the Streamlit code for Tab 1, including the log-scale scatter plot with trend line, the yearly-median comparison of Open-Weight and Closed / API-Only models, the KPIs, the verification panel and the limitations text. It also helped draft the README section and the Part 1 presentation script.	Ran the tab locally on the real Epoch AI data and checked that the model counts matched DATA_DICTIONARY.md (351 models: 207 Open-Weight, 144 Closed / API-Only). Recalculated the displayed 2026 Open-Weight median (4.32×10^{24} FLOP) from its 17 underlying models, and traced a displayed model back to raw_ai_models.csv to compare training FLOP and parameters. Visually reviewed the charts in the dark theme. These checks could have failed if the filtering, aggregation or lookup were wrong.	Counts, the recalculated median and the raw-record values all matched. The visual review found two real problems in the AI-generated version: model labels and the trend line were invisible in dark mode, and the cumulative growth was shown with false precision (×3,298,754). Both were fixed before committing.
				Claude helped write the Streamlit/Plotly code for the compute-vs-score scatter with trend line,		

Qi Wang	Claude (Anthropic)	Development of User Story 1: compute vs capability; main app integration	`tabs/tab2.py`; `app.py`; `data/metadata.json` output in `prepared_data.py`; US1 section of README.md	the compute-band box plot, the KPIs, the deviation tables and the verification panel. It also made the agreed layout changes (insight above the charts, deviation tables collapsed, visible verification, band medians and labelled scores printed on the charts), updated `app.py` to show data coverage and the last-refreshed date, and helped draft the README section. Analytical choices (GPQA Diamond only, $\log_{10}$ compute, at least five results per band, best-version robustness check) were mine.	Ran the full app headlessly on the real data with Streamlit AppTest under both units of analysis. Recomputed the 10^{24}–10^{25} FLOP band (45 results, 26.96%–91.67%) and confirmed the plotted box matches. Traced DeepSeek V4 Flash 0731 (run `EDFusDaaFkh5n2MqKNxMBD`) to `raw_benchmarks.csv`: raw 0.910354 × 100 should equal the displayed 91.04%, and the FLOP (2.496×10^{24}) should match exactly. Re-ran `prepared_data.py` from the raw files and compared the outputs with the committed processed files. These checks could have failed if the filtering, band assignment, score conversion, raw lookup or pipeline were wrong.	All checks matched, and the regenerated `ai_models.csv`, `benchmarks.csv` and `gpqa.csv` were identical to the committed files. Review found one AI-introduced bug: the takeaway box printed the release date twice for models whose names already contain one ("GPT-4 (Jun 2023) (Jun 2023)"). It was fixed before committing.
		Development		Claude wrote most of the Streamlit and Plotly code for Tab 4: the "lowest compute so far" record chart, the dumbbell chart comparing first-to-reach and least-compute models at each level, the KPIs, the access filter	Ran the full app headlessly with Streamlit AppTest on the real data under every access filter and several levels, including the empty state. Traced Grok 4 (run `A85Zfq2qguE4X9xXBweBHP`) to `raw_benchmarks.csv`: the raw score of 0.87	All checks matched ($5 \times 10^{26} \div 2.496 \times 10^{24} = 200.3$). Review found AI-introduced problems that were fixed before committing: a release date printed twice in chart labels

Santosh Pandit	Claude (Anthropic)	of User Story 3A: compute needed to reach a capability level	`tabs/tab4.py` ; US3A section of README.md	and empty-state message, and the raw-data verification panel. It later made the layout changes agreed in the group and helped draft the README section and my Part 1 presentation script. The tab's question (when does a capability level become cheaper to reach?) and the best-run-per-model rule were agreed with the group.	should equal the displayed 87%, and the FLOP (5×10^{26}) should match exactly. Recalculated the headline 200× by dividing Grok 4's raw compute by DeepSeek V4 Flash 0731's (2.496×10^{24}), and confirmed both scores clear the 80% level. Checked the layout at 1366×768. These checks could have failed if the run lookup, score conversion, best-run rule or record logic were wrong.	("Qwen3-30B-A3B (Jul 2025) (Jul 2025)"), KPI model names cut off at 1366×768, the latest chart label clipped at the plot edge, and an answer sentence that claimed "scale leads" even under filters where the data did not support it (it is now generated from the data).
Xingqi Zhang	ChatGPT by OpenAI	Development and debugging of User Story 3B: resource-efficient model choices	`tabs/tab5.py` ; US3B README documentation	ChatGPT assisted with Streamlit code, interactive filters, a ranked dot plot, Pareto-efficient model identification, source verification and debugging.	Ran the dashboard locally using Epoch AI GPQA data. Checked that the raw score of 0.60795455 for gpt-oss-20b converts to 60.80%, and that the raw and processed training compute both equal 5.49e23 FLOP. These checks could have failed if the data lookup or conversion was incorrect.	The score conversion and compute values matched the original benchmark record. The dashboard ran successfully and was committed to GitHub.

Member-Specific AI Use — Nguyen Thao Nhi Truong

Project framing

ChatGPT was used as a discussion and review tool while structuring the project question, target persona and user-story-based division of the dashboard.

The final project framing and allocation of analytical tasks were decided by the group rather than generated automatically.

Data and analytical work

For User Story 2, the analysis uses GPQA Diamond observations prepared from the Epoch AI benchmark data.

The analytical rules applied in the final implementation include:

- using GPQA Diamond observations;
- retaining reported training-compute values;
- excluding missing compute rather than imputing it;

- comparing models within broad one-order-of-magnitude compute bands;
- summarising capability using yearly medians;
- showing interquartile ranges; and
- requiring at least three measured observations before displaying a yearly median.

AI assistance was used to review and implement parts of this workflow, but the resulting values were checked against the processed data.

Visualisation and Streamlit development

ChatGPT was used to assist with debugging and restructuring the Streamlit implementation of User Story 2.

The main visual uses yearly median GPQA Diamond scores with interquartile-range error bars. A scatter plot of individual models was considered as an alternative, but the line chart was retained because the primary task is to compare the year-to-year pattern at comparable compute rather than inspect individual-model variation.

AI assistance was also used to improve the consistency of the page layout with the other dashboard sections.

Verification

The final application contains an explicit verification section.

For the default **10^{24}–10^{25} FLOP** compute band:

- the displayed 2026 median GPQA Diamond score is **89.74%**;
- the result is based on **10 measured observations**;
- the application independently recalculates the median from those observations;
- the recalculated result is also **89.74%**;
- every observation is checked against the selected compute-band boundaries; and
- the displayed observation count is checked against the number of underlying rows.

This verification was designed so that an incorrect filter, aggregation or transformation would produce a visible mismatch rather than being accepted without inspection.

Member-Specific AI Use — Xingqi Zhang (US3B)

Project contribution

I was responsible for developing User Story 3B, which focuses on identifying AI models that achieve strong measured capability with comparatively lower training compute.

My contribution was implemented in `tabs/tab5.py` and integrated into the group's Streamlit application.

ChatGPT was used to assist with understanding the implementation requirements, structuring the dashboard, developing Python and Streamlit code, and troubleshooting technical issues.

Data and analytical work

The analysis uses GPQA Diamond benchmark observations from the Epoch AI dataset, prepared through the group's existing data-processing pipeline.

The final implementation follows these analytical rules:

- Use measured GPQA Diamond benchmark scores.
- Retain observations with positive reported training compute.
- Exclude missing compute values rather than imputing them.
- Apply an adjustable minimum capability threshold.
- Rank qualifying observations by reported training compute.

- Identify Pareto-efficient candidates within the selected comparison group.

The processed dataset contains 316 GPQA observations, of which 93 have usable training-compute information.

ChatGPT assisted with implementing the filtering, ranking and comparison logic. The resulting dashboard was tested locally using the actual project data.

Visualisation and Streamlit development

ChatGPT helped develop the interactive Streamlit interface, including the minimum GPQA score slider, release-year and organisation filters, model ranking visualisation and source verification panel.

The main visualisation is a ranked dot plot using a logarithmic horizontal axis.

A standard linear-axis bar chart was considered as an alternative. However, reported training compute varies by several orders of magnitude, making smaller values difficult to compare on a linear scale.

The dot plot was retained because it provides a clearer comparison of training-compute requirements across qualifying models.

The dashboard also highlights Pareto-efficient candidates to help users examine the trade-off between measured capability and training compute.

Debugging and testing

ChatGPT was used to help resolve Streamlit compatibility and DataFrame display errors encountered during local development.

After modifying the code, I reran the application and checked that the main visualisations, model selection controls and source verification section displayed correctly.

I also tested the minimum GPQA score filter and confirmed that changing the threshold updated the displayed results.

Verification and limitations

The final dashboard includes an explicit source verification section.

Under the default minimum GPQA score threshold of 60%:

- 93 observations have usable training-compute information.
- 43 observations meet the selected performance requirement.
- gpt-oss-20b has the lowest reported training compute among the qualifying observations.

For gpt-oss-20b, the original measured GPQA score is 0.60795455.

Multiplying this value by 100 gives 60.795455%, which rounds to the displayed value of 60.80%.

The original and processed training-compute values both match at 5.49×10^{23} FLOP.

These verification checks could have failed if the benchmark-run lookup, percentage conversion, filtering logic or training-compute transformation were incorrect.

The application displayed matching results for the checked observation.

AI-generated suggestions were evaluated through local code execution and comparison with the source data.

Finally, the analysis does not establish causal algorithmic efficiency, and training compute should not be interpreted as a direct measure of inference cost or real-world operational efficiency.

Member-Specific AI Use — Yuchao Zong (Macro Overview)

Project contribution

I was responsible for the macro overview tab (`tabs/tab1.py`), which shows how quickly the training compute behind notable AI

models has grown since 2018 and compares Open-Weight with Closed / API-Only models. I also wrote the Macro Overview section of the README.

How AI was used

Claude (Anthropic) was used as a coding and writing assistant. It generated most of the Streamlit code for the tab, following the structure and conventions already used in the other tabs, and later made the layout changes agreed in the group (a written takeaway box and a renamed verification section). Claude also helped draft the README section and my Part 1 presentation script.

The analytical rules in the final tab are:

- use `data/ai_models.csv` only, with no imputed compute or parameter values;
- estimate growth with an ordinary least-squares fit of $\log_{10}$(training FLOP) on release date;
- show yearly medians only where a year and access group has at least three models; and
- present the results as descriptive, not causal.

Visualisation decision

The main chart is a scatter plot of every model on a logarithmic compute axis with a fitted trend line. A linear axis was rejected because compute spans more than ten orders of magnitude, so the largest recent runs would flatten every earlier model to the floor.

Verification

- Ran the tab locally on the real data and confirmed the counts against the data dictionary: 351 models, of which 207 are Open-Weight and 144 Closed / API-Only.
- Recalculated the displayed 2026 Open-Weight yearly median (4.32×10^{24} FLOP) from its 17 underlying models; it matched.
- Traced a displayed model back to `data/raw_ai_models.csv` by model name and publication date; training FLOP and parameter count matched exactly.
- Reviewed the rendered charts. This found two problems in the AI-generated code: labels and the trend line were invisible in the dark theme, and the cumulative growth was displayed with false precision (×3,298,754 instead of about 3.3 million times). Both were corrected.

These checks could have failed if the filtering, aggregation, raw-record lookup or display formatting were wrong. I remain responsible for the final code and the claims shown in the tab.

Member-Specific AI Use — Qi Wang (User Story 1 and main app)

Project contribution

I was responsible for User Story 1 (`tabs/tab2.py`), which compares measured GPQA Diamond capability with training compute, and for the main app (`app.py`) that integrates the five tabs, together with deployment.

How AI was used

Claude (Anthropic) was used as a coding and writing assistant. It helped write the Streamlit and Plotly code for the tab, made the layout changes agreed in the group so the tab follows the same structure as the others, updated `app.py` to show data coverage and a "Last refreshed" date read from `data/metadata.json` (written by `prepared_data.py`), and helped draft the US1 section of the README.

The analytical rules in the final tab are:

- use measured GPQA Diamond results with disclosed positive training compute only (93 of 316), with nothing imputed;
- fit an ordinary least-squares trend of score on $\log_{10}$(training FLOP) and report a Spearman rank correlation;
- measure consistency as the score range within one-order-of-magnitude compute bands that contain at least five results;
- offer a best-version-per-model view so repeated evaluations of one model do not drive the result; and
- present the results as an association, not a causal effect.

Visualisation decision

The main chart is a scatter plot on a logarithmic compute axis with a fitted trend line, supported by a box plot of scores within each compute band. A bar chart of average score per band was rejected as the main view because averages would hide the spread between models with comparable compute, which is the point of the user story.

Verification

- Ran the whole app headlessly with Streamlit AppTest on the real data, under both units of analysis, with no errors.
- Recomputed the 10^{24}–10^{25} FLOP band from its 45 underlying results (26.96%–91.67%, median 65.34%); the plotted box matched.
- Traced DeepSeek V4 Flash 0731 to `data/raw_benchmarks.csv` by run ID: the raw score of 0.910354 converts to the displayed 91.04%, and the training FLOP matches exactly.
- Re-ran `prepared_data.py` from the raw files; the regenerated processed files were identical to the committed ones.
- Review found one AI-introduced bug (a release date printed twice in the takeaway box), which was fixed.

These checks could have failed if the filtering, band assignment, score conversion, raw-record lookup or data pipeline were wrong. I remain responsible for the final code and the claims shown in the tab.

Member-Specific AI Use — Santosh Pandit (User Story 3A)

Project contribution

I was responsible for User Story 3A (`tabs/tab4.py`), which asks how much training compute is needed to reach a given GPQA Diamond level and whether that amount has fallen over time. I also wrote the US3A section of the README and aligned the tab with the group's shared layout.

How AI was used

Claude (Anthropic) was used as a coding and writing assistant. It generated most of the Streamlit and Plotly code for the tab, following the conventions of the other tabs (shared colour palette, widget keys prefixed `tab4_` , an entry function called by `app.py`), and later restructured the page to the agreed layout: user story, scope caption, controls, KPIs, insight, charts, visible verification, and limitations in an expander. Claude also helped draft the README section and my Part 1 presentation script.

The analytical rules in the final tab are:

- use measured GPQA Diamond results with positive reported training compute only, with nothing imputed;
- represent each model by its best-scoring run (the same rule as User Story 1);
- start capability levels at 40%, well above the 25% chance level;
- define a "record" as a model reaching the level with less compute than every earlier qualifying model; and
- present the results as associations, not causal effects.

Visualisation decision

Chart 2 is a dumbbell chart on a logarithmic compute axis. A grouped bar chart was rejected because compute spans about four orders of magnitude: on a linear axis smaller models' bars would be invisible, and on a log axis bars do not start at zero, so bar length would mislead.

Verification

- Ran the whole app headlessly with Streamlit AppTest on the real data, under all three access filters and several levels, with no errors; confirmed that asking for an open-weight model at 95% shows the empty-state message (the best open-weight score is 93.1%).
- Traced Grok 4 (run `A85Zfq2qguE4X9xXBweBHP`) to `data/raw_benchmarks.csv` : the raw score of 0.87 converts to the displayed 87%, and the training FLOP (5×10^{26}) matches exactly.
- Recalculated the headline reduction from the raw file: $5 \times 10^{26} \div 2.496 \times 10^{24}$ (DeepSeek V4 Flash 0731) = 200.3, matching the displayed 200×.
- Checked the layout at 1366×768 with vertical scrolling only.

- Review found and fixed several AI-introduced issues: duplicated release dates in labels, truncated KPI labels, a clipped chart label, and an over-general answer sentence that is now generated from the data.

These checks could have failed if the run lookup, score conversion, best-run rule, record logic or display formatting were wrong. I remain responsible for the final code and the claims shown in the tab.

Responsibility for Final Outputs